13 · Revision set (the spaced re-test)

One question per key idea. Do this on paper, without the pack, a day or more after finishing the chapters, and again two or three days later. Typing the programs from memory is the real test.

> **HOW TO USE IT**
>
> Answer everything first, then mark with the answers at the end. Anything you missed: re-read that chapter section, then **type that program from scratch** the same day. Don't just re-read: being able to type it without looking is what earns marks.

Part A — type from memory (the actual exam skill)

Close the pack, open your editor, and type each program until it compiles and gives the expected output (chapter 12.2). Time yourself; aim for under 15 minutes each.

1. Exp 3 (insert, all three cases)
2. Exp 4 (delete, all three cases)
3. Exp 5 (matrix multiply with the three functions)
4. Exp 7 (insertion sort)
5. Exp 8 (quick sort with partition)
6. Exp 10 (queue with menu)
7. Exp 11 (singly linked list with menu)
8. Exp 12 (circular linked list with menu)

Part B — short questions

1. `int x = 9; int *p = &x; *p = 4;`. What is x now?
2. `int a[5]`; `int *p = a;`. Write the address of `a[3]` using p, and its value.
3. Insert at position 2 in `3 6 9` (n = 3), value 4. Final array?
4. Why does the insertion shift loop run from the end, but the deletion shift loop from the front?
5. Valid positions for insertion in an n-element array? For deletion?
6. A is 3 × 2, B is 2 × 4. Is A × B possible? Order of the result? Number of multiplications?
7. Transpose of a 4 × 1 matrix: what order?
8. Insertion sort on `3 1 2`: write the array after each pass.
9. Best-case input for insertion sort, and its time complexity?
10. Partition `7 2 9 4` with the last element as pivot. Final array and returned index?
11. Which input makes this program's quick sort $O(n^2)$?
12. Why does `swapElements` take `int &a, int &b`?
13. Linear queue, MAX = 3: enqueue 1, 2, 3, dequeue, enqueue 4. What happens, and why?
14. Circular queue condition for "full"?
15. Stack: push 1, 2, 3, pop, push 4, pop, pop. Which values come out?
16. In a singly linked list, how do you know you're at the last node? In a circular one?
17. What goes wrong if traverse forgets `temp = temp->next;`?
18. In delete-from-beginning, what goes wrong with `delete head; head = head->next;`?
19. Why is delete-from-beginning O(1) in a singly list but O(n) in the circular list (as programmed)?
20. Why is `int arr[n];` (n read with cin) a problem?
21. What does `p->data` mean, written another way?
22. Give the stability (stable or not) of insertion sort and of quick sort.

Answers

1. **4**: `*p = 4` writes through the pointer into x.
2. Address `p + 3`; value `*(p + 3)` (same as `a[3]`).
3. **3 4 6 9**.
4. In each direction, the element about to be overwritten must be copied first: shifting right, the last element moves first; shifting left, the one just after the gap moves first.
5. Insertion: 1 … n+1. Deletion: 1 … n.
6. Yes (2 = 2). Result 3 × 4. Multiplications: 3 · 4 · 2 = **24**.
7. **1 × 4**.
8. Pass 1 (key 1): `1 3 2`. Pass 2 (key 2): `1 2 3`.
9. An already sorted array: **O(n)** (n − 1 comparisons).
10. pivot 4, i = −1. j = 0: 7 no. j = 1: 2 < 4 → i = 0, swap(arr[0], arr[1]) → `2 7 9 4`. j = 2: 9 no. Final swap(arr[1], arr[3]) → **`2 4 9 7`**, returns **1**.
11. An already sorted (or reverse-sorted) array: the last-element pivot is always the largest (or smallest).
12. So the function swaps the real array elements, not copies.
13. **Overflow** on enqueue 4: rear is still MAX − 1 even though slot 0 was freed. This is the linear-queue flaw.
14. `(rear + 1) % MAX == front`.
15. **3, 4, 2** (LIFO); 1 remains.
16. Singly: `temp->next == NULL`. Circular: `temp->next == head`.
17. `temp` never moves, so the loop never ends (infinite loop).
18. After `delete head`, reading `head->next` uses freed memory; the rest of the list is lost or the program crashes. Save `temp = head`, move head, then delete temp.
19. In the circular list the last node must be re-pointed to the new head, and finding it means walking the whole list.
20. It's a variable-length array: not standard C++. Visual Studio rejects it and strict g++ errors on it. Use a fixed maximum like `int arr[100]`.
21. `(*p).data`.
22. Insertion sort: **stable**. Quick sort: **not stable**.